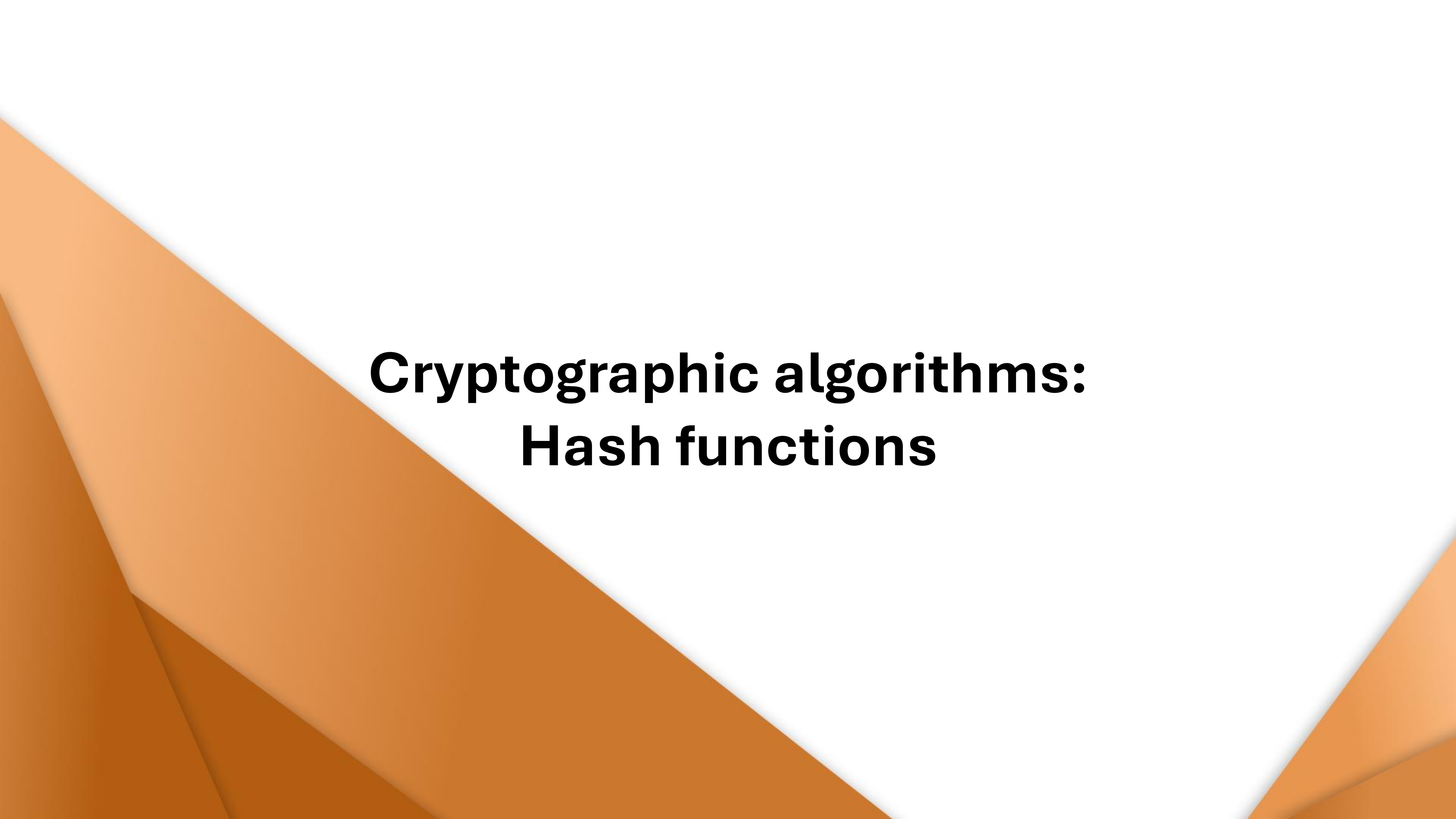




Cryptographic algorithms: Hash functions

Learning objectives

- **Explain what a hash function is**
- **Learn how hashes are used in cybersecurity**
- **Explore examples like password protection and data integrity**

What is a hash function?

- **Converts data into fixed length output**
- **Input and output are always the same length**
- **One-way process that cannot be reversed**

Key properties of hash functions

- **Deterministic and fast**
- **Irreversible and collision resistant**
- **Avalanche effect: small input change causes large output change**

How hashing differs from encryption

- **Hashing has no key and cannot be reversed**
- **Used for verification, not confidentiality**
- **Helps detect tampering or errors**

Use case: password storage

- **Store only hashed version of passwords**
- **User input is hashed and compared with stored hash**
- **Prevents original passwords from being exposed**

Use case: data integrity

- **Hashes verify downloaded or received files**
- **Mismatch indicates tampering or corruption**
- **Used in software, backups and messages**

Common hash algorithms



**Fast but
not secure**



**Outdated
and weak**



**Modern and
trusted**

Example in Python

- **Uses hashlib module to create hash**
- **Text is encoded and hashed with SHA 256**
- **Output is a fixed hexadecimal string**

```
import hashlib

text = "Hello"
hashed =
hashlib.sha256(text.encode()).hexdigest()
print(hashed)
```

Strengths and weaknesses

- **Hash functions are fast and simple**
- **Weak hashes are vulnerable to attacks**
- **Use salting and slow hashing for passwords**

Summary

- **Hashing ensures data has not changed**
- **Essential for verifying data and protecting passwords**

Next

- **Learn how digital signatures use hashing and keys**